

PHP 개발자를 위한 Python 프로그래밍

TDD 기반 중급 개발자 맞춤 — 정적/동적 UML 포함

automater 코드베이스의 실제 패턴으로 배우는 Python 3.12+

이 교재는 TDD 방식으로 대형 풀스택 프로젝트를 성공적으로 개발한 경험이 있는 PHP 중급 개발자를 위한 Python 입문서입니다. Robert C. Martin의 UML 철학("코드의 지름길")에 따라, 정적/동적 UML 다이어그램으로 Python의 객체지향 구조를 PHP와 대비하여 설명합니다. 모든 예제는 automater 코드베이스의 실제 코드에서 가져왔습니다.

- PHP 8.0+ (union types, match expression, readonly properties)
- PHPUnit으로 TDD 경험 (Mock, Spy, Stub 이해)
- 인터페이스, 추상 클래스, 의존성 주입 패턴 이해
- 정적 UML (Class Diagram) 및 동적 UML (Sequence Diagram) 해석 가능

목차

1. PHP → Python 기초 문법 매핑

- 1.1 변수와 타입
- 1.2 함수와 타입 힌트
- 1.3 문자열과 f-string
- 1.4 컬렉션: list, dict, tuple
- 1.5 제어 흐름: if, for, match/case

2. 클래스와 객체지향

- 2.1 class: PHP class → Python class
- 2.2 dataclass: PHP readonly class → Python frozen dataclass
- 2.3 ABC: PHP interface → Python ABC
- 2.4 상속과 다형성 — UML로 보는 구조

3. 타입 시스템 심화

- 3.1 Union, Literal, Optional
- 3.2 type alias (PEP 695)
- 3.3 Generic과 Protocol

4. pytest로 TDD

- 4.1 PHPUnit → pytest 매핑
- 4.2 fixture와 DI
- 4.3 Test Double 전략 — Stub, Spy, Fake
- 4.4 UML로 보는 RecordingEditor Spy 패턴

5. 디자인 패턴 in Python

- 5.1 Command: PostStep.execute() — UML 포함
- 5.2 Strategy: BlockHandler + Registry — UML 포함
- 5.3 Factory Method: create_block() — UML 포함

6. Python 고유 패턴

- 6.1 제너레이터와 yield
- 6.2 컨텍스트 매니저 (with)
- 6.3 데코레이터
- 6.4 Pathlib

7. SQLAlchemy 2.0 → Eloquent 매핑

- 7.1 모델 정의
- 7.2 쿼리 빌더
- 7.3 관계 (relationship)

8. 실전: automater 코드 읽기 가이드

1. PHP → Python 기초 문법 매핑

PHP 개발자가 Python 코드를 처음 읽을 때 가장 혼란스러운 점은 세미콜론 없음, 종괄호 대신 들여쓰기, \$ 없는 변수입니다. 이 장은 이 "시각적 충격"을 빠르게 극복합니다.

1.1 변수와 타입

<pre>// PHP \$name = "hello"; // string \$count = 42; // int \$rate = 3.14; // float \$active = true; // bool \$items = null; // null</pre>	<pre># Python name = "hello" # str count = 42 # int rate = 3.14 # float active = True # bool (True!) items = None # None (None!)</pre>
---	--

변수 선언: \$ 없음, ; 없음, True/False/None은 대문자

Python은 동적 타입이지만, 타입 힌트를 적극 사용합니다. PHP 8.0+의 타입 선언과 유사하지만 런타임에 강제되지 않고, mypy 같은 정적 분석기가 검사합니다.

1.2 함수와 타입 힌트

<pre>// PHP function greet(string \$name): string { return "Hello, {\$name}"; } // PHP function add(int \$a, int \$b = 0): int { return \$a + \$b; }</pre>	<pre># Python def greet(name: str) -> str: return f"Hello, {name}" # Python def add(a: int, b: int = 0) -> int: return a + b</pre>
---	---

함수: def, 콜론으로 블록 시작, return 타입은 -> 뒤에

automater 실제 코드에서 확인:

```
# automator/title_generator.py
def generate_title(option: TitleOption, rng: random.Random | None = None) -> str:
    if option.fixed_title:
        return option.fixed_title
    ...
```

| (pipe) 문법: Python 3.10+에서 Union[X, Y] 대신 X | Y를 쓸 수 있습니다. PHP 8.0의 string|null과 동일합니다.

1.3 문자열과 f-string

<pre>// PHP \$kw = "key value"; \$msg = "{\$kw}() ()"; // PHP \$msg = "\$kw" . "() ()";</pre>	<pre># Python kw = "key value" msg = f"{kw}() ()" # Python 3.12+: f-string log = f"keyword='{kw}'"</pre>
---	--

f-string: PHP의 "{\$var}" → Python의 f"{var}"

1.4 컬렉션: list, dict, tuple

<pre>// PHP // Array (PHP 배열) \$items = ["a", "b", "c"]; \$map = ["key" => "value"]; // PHP \$first = \$items[0]; \$val = \$map["key"];</pre>	<pre># Python # list (가변 배열) items = ["a", "b", "c"] # dict (key-value 매핑) map = {"key": "value"} # tuple (불변 튜플) coords = (1, 2, 3) first = items[0] val = map["key"]</pre>
---	---

PHP array → Python list(가변) + dict(매핑) + tuple(불변)

핵심 차이	PHP의 array는 list와 dict를 겸합니다. Python은 이 둘을 명확히 분리합니다. automater에서 tuple은 불변 데이터에 사용됩니다: PostingSpec.body: tuple[Section, ...]. 이것은 PHP의 readonly property와 같은 의도입니다.
---------------------	---

1.5 제어 흐름: if, for, match/case

```
// PHP
// if
if ($x > 0) {
    echo "positive";
} elseif ($x == 0) {
    echo "zero";
}

// for
foreach ($items as $item) {
    echo $item;
}

// match (PHP 8.0+)
$result = match($status) {
    "active" => true,
    "disabled" => false,
    default => null,
};
```

```
# Python
# if (조건식 then!)
if x > 0:
    print("positive")
elif x == 0:
    print("zero")

# for
for item in items:
    print(item)

# match/case (Python 3.10+)
match status:
    case "active":
        result = True
    case "disabled":
        result = False
    case _:
        result = None
```

match/case: PHP 8.0 match → Python 3.10 match/case (더 강력한 패턴 매칭)

1.6 PHP 개발자가 흔히 빠지는 함정 5가지

함정 1: 들여쓰기가 문법이다

PHP에서 들여쓰기는 가독성을 위한 것이지만, Python에서는 블록을 정의하는 문법입니다. 탭과 스페이스를 섞으면 `IndentationError`가 발생합니다. 팀 표준: 스페이스 4칸. `.editorconfig`에 `indent_style = space, indent_size = 4`를 설정하세요.

함정 2: 가변 기본값 공유 버그

<pre>// PHP // PHP - [] function add(array \$items = []) { \$items[] = "new"; return \$items; } add(); // ["new"] add(); // ["new"] ← [] 가 공유</pre>	<pre># Python # Python - []! def add(items=[]): # [] 가 공유! items.append("new") return items add() # ["new"] add() # ["new", "new"] ← [] 가 공유! # None 처리 def add(items=None): items = items or [] items.append("new") return items</pre>
---	--

Python 함정: 가변 기본값(list, dict)은 함수 정의 시 한 번만 생성되어 모든 호출이 공유

함정 3: Truthiness (참/거짓 판정)

PHP	Python	판정
0, 0.0, "", "0", [], null, false	0, 0.0, "", [], {}, (), None, False	Falsy
"0" → false (PHP !!)	"0" → True (Python !!)	핵심 차이!
empty(\$var)	not var (len(var)==0)	빈 값 체크

Table 1-1. Truthiness 차이 — PHP의 "0"은 false, Python의 "0"은 True

함정 4: == vs is

```
# == 비교 (PHP ==, Python ==)
a = [1, 2, 3]; b = [1, 2, 3]
a == b # True (PHP, Python)

# is 비교 (PHP is, Python is)
a is b # False (PHP, Python)

# None 처리 (PHP ===, Python is)
if value is None: # Python
if value == None: # PHP
```

함정 5: List Comprehension (한 줄 반복문)

<pre>// PHP // PHP - array_map + [] \$squares = array_map(fn(\$x) => \$x ** 2, [1, 2, 3, 4, 5]); \$evens = array_filter(\$numbers, fn(\$x) => \$x % 2 === 0);</pre>	<pre># Python # Python - list comprehension squares = [x ** 2 for x in [1,2,3,4,5]] evens = [x for x in numbers if x % 2 == 0] # dict comprehension slug_map = {kw.category.slug: kw.value for kw in combo.keywords} # automater 블록 (layout.py): return [block for section in sections for block in section.blocks]</pre>
--	---

array_map/filter → list comprehension: Python에서 가장 많이 쓰는 패턴

1.7 에러 처리: try/catch → try/except

<pre>// PHP // PHP try { \$result = riskyOperation(); } catch (ValueError \$e) { echo \$e->getMessage(); } catch (RuntimeException \$e) { log(\$e); throw \$e; // re-throw } finally { cleanup(); }</pre>	<pre># Python # Python try: result = risky_operation() except ValueError as e: print(e) except RuntimeError as e: log(e) raise # re-raise (?? ??) finally: cleanup() # Python 3.11+: ExceptionGroup except* ValueError as eg: # ?? ValueError? ? ?? ??</pre>
--	---

catch → except, throw → raise, throw \$e → raise (인자 없이)

automater 실제 패턴 — BatchWorker의 에러 격리:

```
# factory/batch_worker.py - _process_item()
def _process_item(self, item, batch) -> ItemResult:
try:
item.status = "running"
spec = build_posting_spec(combo=item.combination, ...)
editor = self._editor_factory(batch.account)
self._runner.run(spec, editor)
item.status = "completed"
return ItemResult(item_id=item.id, success=True)
except Exception as exc:
item.status = "failed"
item.error_message = str(exc)
return ItemResult(item_id=item.id, success=False, error=str(exc))
```

PHP에서 catch(\Exception \$e)로 모든 예외를 캐치하는 것과 동일합니다. except Exception은 KeyboardInterrupt, SystemExit 등 시스템 예외를 제외한 모든 예외를 캐치합니다.

1.8 모듈과 import: PHP namespace → Python 패키지

<pre>// PHP // PHP - namespace + use namespace App\Automator; use App\Automator\Contracts\PostingSpec; use App\Automator\Ports\TextGenerator; class JobRunner { // PostingSpec? ??? use? import }</pre>	<pre># Python # Python - ??? + import # automator/runner.py from automator.contracts import PostingSpec from automator.ports import TextGenerator class JobRunner: # PostingSpec? ??? import pass # __init__.py? ?????? ?????? ?? # (PHP? composer.json autoload? ??)</pre>
---	---

namespace → 패키지(디렉토리), use → from ... import, autoload → __init__.py

<pre>from __future__ import annotations</pre>	automater의 모든 파일 첫 줄에 이 import가 있습니다. 이것은 타입 힌트를 문자열로 평가하여 순환 참조를 방지합니다. PHP에서 클래스가 아직 정의되지 않은 타입을 참조할 때 발생하는 문제와 같습니다. Python 3.14에서는 이 import 없이 기본 동작이 됩니다 (PEP 649).
---	---

2. 클래스와 객체지향

2.1 class: PHP class → Python class

```
// PHP
class Account {
    public function __construct(
        private string $username,
        private string $password,
        private array $meta = [],
    ) {}

    public function getUsername(): string {
        return $this->username;
    }
}

# Python
class Account:
    def __init__(
        self,
        username: str,
        password: str,
        meta: dict | None = None,
    ) -> None:
        self.username = username
        self.password = password
        self.meta = meta or {}

    # property
```

생성자: PHP __construct → Python __init__, \$this → self

핵심 차이: PHP는 constructor promotion(private string \$username)으로 프로퍼티 선언과 할당을 동시에 합니다. Python은 __init__에서 self.username = username을 명시해야 합니다. 하지만 dataclass가 이 boilerplate를 해결합니다.

2.2 dataclass: PHP readonly class → frozen dataclass

```
// PHP
// PHP 8.2+
readonly class AccountOption {
    public function __construct(
        public string $username,
        public string $password,
        public array $meta = [],
        public array $proxies = [],
        public string $sessionPath = "",
    ) {}
}

# Python
# automator/options.py
@dataclass(frozen=True)
class AccountOption:
    username: str
    password: str
    meta: dict = field(default_factory=dict)
    proxies: list[str] = field(default_factory=list)
    session_path: str = ""
```

readonly class → @dataclass(frozen=True): 생성 후 변경 불가

field(default_factory=dict): PHP에서 = []은 안전하지만, Python에서 가변 기본값(dict, list)은 모든 인스턴스가 공유하는 버그를 만듭니다. default_factory는 매번 새 인스턴스를 생성합니다.

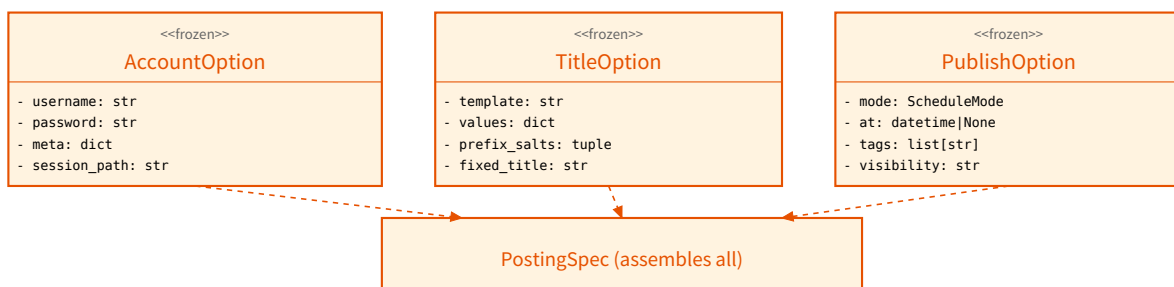


Fig 2-1. Value Objects — 모든 Option은 frozen dataclass (PHP readonly class 대응)

2.3 ABC: PHP interface → Python ABC

```
// PHP
interface TextGenerator {
    /**
     * @return string[]
     */
    public function generate(
        string $prompt,
        int $count
    ): array;
}

# Python
# automator/ports.py
from abc import ABC, abstractmethod

class TextGenerator(ABC):
    @abstractmethod
    def generate(
        self,
        prompt: str,
        count: int
    ) -> list[str]:
        ...
```

interface → ABC + @abstractmethod: 인스턴스화 시도 시 TypeError 발생

```
// PHP
// []
class StubTextGenerator implements TextGenerator {
    public function generate(
        string $prompt, int $count
    ): array {
        return array_slice($this->stubs, 0, $count);
    }
}
```

```
# Python
# automator/stubs.py [] []
class StubTextGenerator(TextGenerator):
    def __init__(self, paragraphs=None):
        self._paragraphs = paragraphs or [...]

    def generate(self, prompt: str, count: int) -> list[str]:
        return [self._paragraphs[i % len(self._paragraphs)]
                for i in range(count)]
```

implements → 괄호 안에 부모 클래스: class Sub(Parent)

2.4 상속과 다형성 — UML로 보는 구조

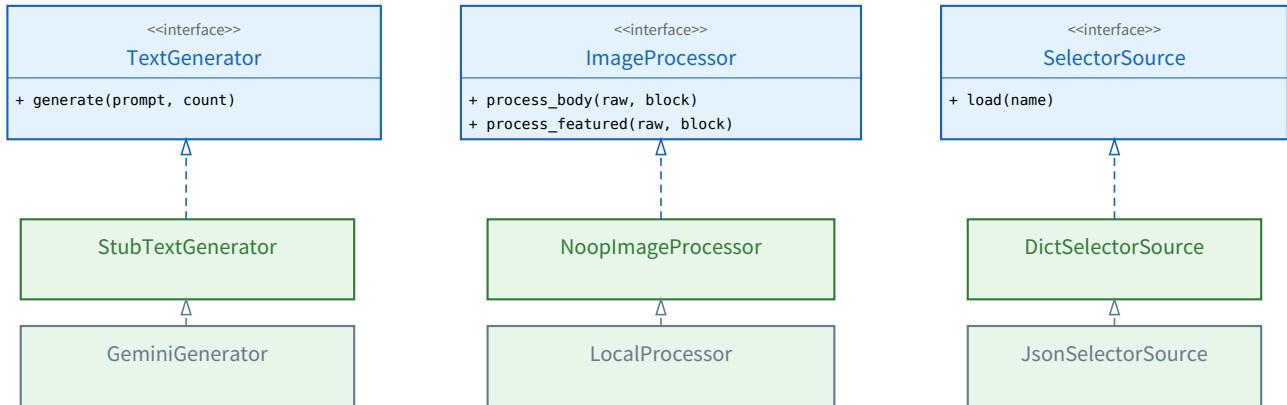


Fig 2-2. Port ABCs — PHP interface와 동일한 역할, 테스트 시 Stub 주입

이 구조는 PHP의 interface + DI Container와 정확히 동일합니다. StubTextGenerator는 PHPUnit의 `$this->createMock(TextGenerator::class)`과 같은 역할이지만, 직접 작성된 test double입니다. mock.patch 없이 생성자 주입만으로 테스트합니다.

2.5 @property와 매직 메서드

```
// PHP
// PHP - getter/setter
class Account {
    public function __construct(
        private string $sessionPath = ""
    ) {}

    public function getResolvedSessionPath(): string {
        return $this->sessionPath
        ?: "{$this->username}_session.json";
    }

    public function __toString(): string {
        return $this->username;
    }
}
```

```
# Python
# automator/options.py [] []
@dataclass(frozen=True)
class AccountOption:
    username: str
    session_path: str = ""

    @property
    def resolved_session_path(self) -> str:
        return self.session_path or \
            f"{self.username}_session.json"

    def session_exists(self) -> bool:
        return Path(
            self.resolved_session_path).exists()

    def __str__(self) -> str:
        return self.username
```

@property: PHP getter → Python @property (메서드를 속성처럼 접근)

@property는 `obj.resolved_session_path`로 접근할 수 있게 합니다 (괄호 없이). PHP의 `getResolvedSessionPath()`에 해당하지만, 호출 문법이 속성처럼 자연스럽습니다. frozen dataclass와 함께 쓰면 '계산된 읽기 전용 속성'이 됩니다.

PHP	Python	용도
<code>__construct()</code>	<code>__init__()</code>	생성자 (객체 초기화)
<code>__toString()</code>	<code>__str__()</code>	문자열 변환 (echo/print)
<code>__get()/__set()</code>	<code>@property / @x.setter</code>	속성 접근 제어
<code>__clone()</code>	<code>__copy__() / __deepcopy__()</code>	객체 복사
<code>__invoke()</code>	<code>__call__()</code>	객체를 함수처럼 호출
<code>Countable::count()</code>	<code>__len__()</code>	<code>len(obj)</code> 지원
<code>Iterator::current()/next()</code>	<code>__iter__() / __next__()</code>	for 루프 지원

ArrayAccess::offsetGet()	__getitem__()	obj[key] 지원
JsonSerializable	__dict__ [] asdict()	직렬화

Table 2-1. PHP 매직 메서드 → Python dunder(더블 언더스코어) 메서드 매핑

2.6 가시성: public/protected/private

```
// PHP
// PHP - [] [] []
class Foo {
    public string $name;
    protected int $count;
    private array $data;

    private function helper(): void {}
}
```

```
# Python
# Python - [] [] []
class Foo:
    def __init__(self):
        self.name = "public" # public
        self._count = 0 # protected ([])
        self.__data = [] # private (name mangling)

    def _helper(self): # protected ([])
        pass
```

Python에는 접근 제어자가 없음. _단일언더스코어 = protected 관례, __이중 = name mangling

automater 코드에서 확인: ContentBuilder의 _text_gen, _img_proc은 _ prefix입니다. 이것은 "외부에서 직접 접근하지 마세요"라는 관례이지, 실제로 접근을 막지는 않습니다. Python 철학: "We're all consenting adults here." (우리는 모두 성인이다)

3. 타입 시스템 심화

3.1 Union, Literal, Optional

```
// PHP
// PHP 8.0+ union types
function find(int|string $id): ?User {
// ?User === User|null
...
}

// PHP 8.1 enum
enum Visibility: string {
case Public = "public";
case Private = "private";
}
```

```
# Python
# Python 3.10+ pipe syntax
def find(id: int | str) -> User | None:
...

# Literal (enum [], [])
from typing import Literal
Visibility = Literal["public", "private", "draft"]

# automator/options.py [] []
BlockRole =
Literal["intro", "body", "supporting", "cta", "closing"]
BlockTone =
Literal["informational", "review", "story", "promotional"]
```

union types: PHP int|string → Python int | str, PHP enum → Python Literal

Literal vs Enum: Python에도 enum.Enum이 있지만, automater는 Literal을 선호합니다. 이유: frozen dataclass의 필드 타입으로 사용할 때 직렬화/역직렬화가 단순하고, JSON config와 1:1 대응이 됩니다.

3.2 type alias (PEP 695, Python 3.12+)

```
// PHP
// PHP type alias [] []
// PHPStan/Psalm [] @psalm-type [] []

/** @psalm-type Block = HeadingBlock
 * | ParagraphBlock | ImageBlock
 * | FeaturedImageBlock | ListBlock
 * | QuoteBlock | DividerBlock
 */
```

```
# Python
# automator/options.py [] []
from typing import Union

Block = Union[
    HeadingBlock,
    ParagraphBlock,
    ImageBlock,
    FeaturedImageBlock,
    ListBlock,
    QuoteBlock,
    DividerBlock,
]

# Python 3.12+ [] []:
# type Block = HeadingBlock | ParagraphBlock | ...
```

sealed type alias: PHP PHPStan @psalm-type → Python Union / type 문

3.3 Generic과 Protocol

```
// PHP
// PHP generics [] [] (PHPStan [])
/** @template T */
/** @param T[] $items */
/** @return T */
function first(array $items) {
return $items[0];
}
```

```
# Python
# Python generics
from typing import TypeVar
T = TypeVar("T")
def first(items: list[T]) -> T:
return items[0]

# Python 3.12+ [] []:
def first[T](items: list[T]) -> T:
return items[0]
```

generics: PHP PHPStan @template → Python TypeVar (3.12+: 함수 시그니처에 직접)

4. pytest로 TDD

4.1 PHPUnit → pytest 매핑

PHPUnit	pytest	automater 실제 사용
class FooTest extends TestCase	class TestFoo: (()) (())	class TestBuildValues:
public function testBar()	def test_bar(self):	def test_inactive_suffix_stripped(self):
\$this->assertEquals(a, b)	assert a == b	assert values['region'] == '[]'
\$this->assertInstanceOf(X, obj)	assert isinstance(obj, X)	assert isinstance(gen, TextGenerator)
\$this->expectException(E)	with pytest.raises(E):	with pytest.raises(ValueError):
setUp() / tearDown()	@pytest.fixture	@pytest.fixture\ndef runner():
@dataProvider	@pytest.mark.parametrize	(() () ())
createMock(Interface)	() Stub/Fake ()	StubTextGenerator()

Table 4-1. PHPUnit → pytest 매핑 — automater 코드베이스 기준

4.2 fixture와 DI

```
// PHP
// PHPUnit
class RunnerTest extends TestCase {
    private Runner $runner;

    protected function setUp(): void {
        $textGen = new StubTextGenerator();
        $imgProc = new NoopImageProcessor();
        $this->runner = new JobRunner(
            new SpecValidator(),
            new ContentBuilder($textGen, $imgProc)
        );
    }
}

# Python
# tests/integration/automator/test_runner.py
@pytest.fixture
def runner():
    text_gen = StubTextGenerator()
    img_proc = NoopImageProcessor()
    return JobRunner(
        SpecValidator(),
        ContentBuilder(text_gen, img_proc)
    )

def test_run_calls_open(runner, rec):
    runner.run(_spec(), rec)
    assert rec.calls[0] == ("open",)
```

setUp() → @pytest.fixture: 함수 인자에 이름만 쓰면 자동 주입

4.3 Test Double 전략 — Stub, Spy, Fake

automater는 mock.patch를 거의 사용하지 않습니다. Port ABC가 의존성 주입의 경계를 제공하므로, 생성자 주입만으로 test double을 교체합니다. PHP에서 Mockery/PHPUnit Mock 대신 직접 Fake 클래스를 작성하는 것과 같습니다.

```
# automator/stubs.py - () () test double
class StubTextGenerator(TextGenerator): # Stub: () ()
    def generate(self, prompt, count): return self._paragraphs[:count]

class NoopImageProcessor(ImageProcessor): # Noop: () () () ()
    def process_body(self, raw, block): return raw

class DictSelectorSource(SelectorSource): # Fake: () () () ()
    def load(self, name): return self._loaders[name]
```

4.4 UML로 보는 RecordingEditor Spy 패턴

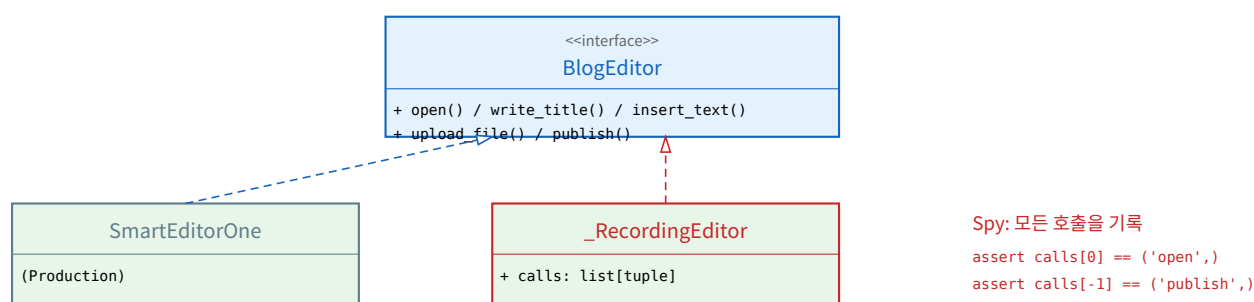


Fig 4-1. RecordingEditor (Spy) — BlogEditor ABC를 구현하여 호출 순서를 기록

PHP 대응: PHPUnit의 `$this->spy = $this->createMock(BlogEditor::class) + $spy->expects($this->exactly(3))`. Python에서는 직접 클래스를 작성하여 calls 리스트에 호출을 기록합니다. mock.patch보다 명시적이고, import 경로 변경에 영향받지 않습니다.

4.5 conftest.py와 테스트 격리

PHP에서 `setUp()/tearDown()`은 각 테스트 클래스에 속합니다. Python pytest에서는 `conftest.py`에 fixture를 정의하면 같은 디렉토리의 모든 테스트가 공유합니다. PHP의 TestCase 부모 클래스에 공통 setUp을 넣는 것과 유사하지만, 상속 없이 동작합니다.

```
# tests/integration/factory/conftest.py
@pytest.fixture(scope="session")
def engine():
    eng = create_engine(test_url)
    create_schema(eng) # CREATE TABLE
    yield eng # ← DB 생성 및 초기화
    drop_schema(eng) # DROP TABLE
    eng.dispose()

@pytest.fixture(scope="function")
def session(engine) -> Session:
    with Session(engine) as sess:
        yield sess
    sess.rollback() # ← DB 롤백
```

scope="session": 전체 테스트 세션에서 한 번만 실행 (PHP의 setUpBeforeClass). scope="function": 매 테스트마다 실행 (PHP의 setUp). yield: 제너레이터 기반 fixture. yield 전이 setup, yield 후가 teardown입니다. 세션 롤백으로 테스트 간 DB 격리를 보장합니다 — PHP Laravel의 RefreshDatabase 트레이트와 동일한 역할입니다.

4.6 parametrize: 하나의 테스트로 여러 케이스

<pre>// PHP // PHPUnit @dataProvider class TitleTest extends TestCase { public static function templates(): array { return [['', ValueError::class], ['{}', ValueError::class], ['{a} {b}', null], // OK]; } } /** @dataProvider templates */ public function testValidate(string \$tpl, ?string \$exc): void { if (\$exc) \$this->expectException(\$exc); validate_template(\$tpl); } }</pre>	<pre># Python # pytest parametrize import pytest @pytest.mark.parametrize("tpl, should_raise", [('', True), ('{}', True), ('{a} {b}', False),]) def test_validate_template(tpl, should_raise): if should_raise: with pytest.raises(ValueError): validate_template(tpl) else: validate_template(tpl) # no error</pre>
--	---

@dataProvider → @pytest.mark.parametrize: 데코레이터로 케이스 목록 전달

4.7 SimpleNamespace Fake: DB 없이 ORM 객체 흉내내기

automater의 단위 테스트는 DB를 사용하지 않습니다. ORM 객체 대신 SimpleNamespace로 가벼운 fake를 만듭니다. PHP에서 (object) ["key" => "value"]로 stdClass를 만드는 것과 유사합니다.

```
# tests/unit/factory/test_job_builder.py
from types import SimpleNamespace

def _keyword(slug, value, cat_id=1, affixes=None):
    category = SimpleNamespace(slug=slug, id=cat_id)
    return SimpleNamespace(
        category=category, value=value, affixes=affixes or []
    )

# ORM Keyword 객체 흉내내기
kw = _keyword("region", "지역")
kw.category.slug # "region"
kw.value # "지역"
kw.affixes # []
```

이 패턴의 핵심: `job_builder._build_values(combo)`는 `combo.keywords`를 순회하며 `kw.category.slug`와 `kw.value`에 접근합니다. `SimpleNamespace`가 이 속성들을 제공하므로 실제 ORM 객체와 동일하게 동작합니다. Duck Typing의 힘입니다.

5. 디자인 패턴 in Python

5.1 Command: PostStep.execute()

Martin은 시퀀스 다이어그램에서 "인터페이스로 메시지를 보내라"고 강조합니다. PostStep은 자기 자신을 실행하는 Command 객체이며, JobRunner는 step의 구체 타입을 알 필요 없이 execute(editor)만 호출합니다.

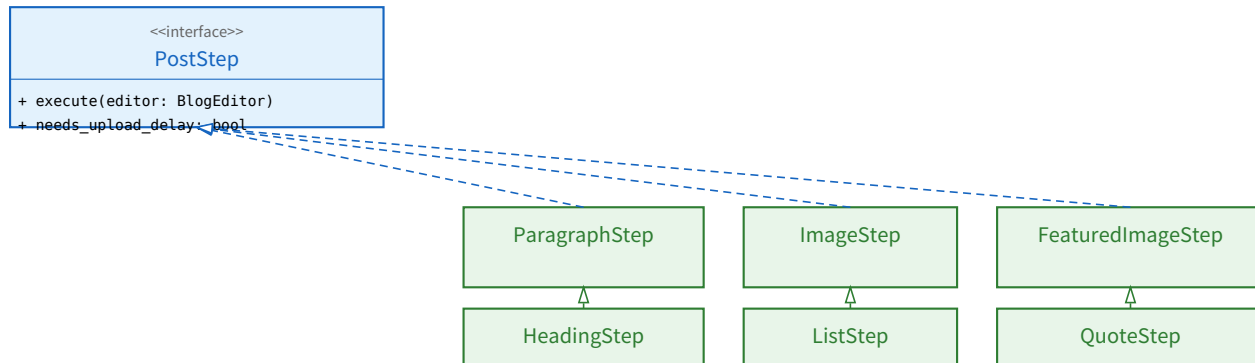


Fig 5-1. PostStep Command 계층 — PHP의 interface Command { execute(); } 와 동일

<pre>// PHP // PHP Command Pattern interface PostStep { public function execute(BlogEditor \$editor): void; } class ParagraphStep implements PostStep { public function __construct(private readonly string \$text, private readonly int \$newlines = 2) {} public function execute(BlogEditor \$editor): void { \$editor->insertText(\$this->text, \$this->newlines); } }</pre>	<pre># Python # automator/editor.py class PostStep(ABC): @abstractmethod def execute(self, editor: BlogEditor) -> None: ... @property def needs_upload_delay(self) -> bool: return False @dataclass(frozen=True) class ParagraphStep(PostStep): text: str newlines: int = 2 def execute(self, editor: BlogEditor) -> None: editor.insert_text(self.text, self.newlines)</pre>
---	---

Python은 @dataclass + ABC 상속을 조합하여 데이터와 행위를 하나의 클래스에 담음

5.2 Strategy: BlockHandler + Registry

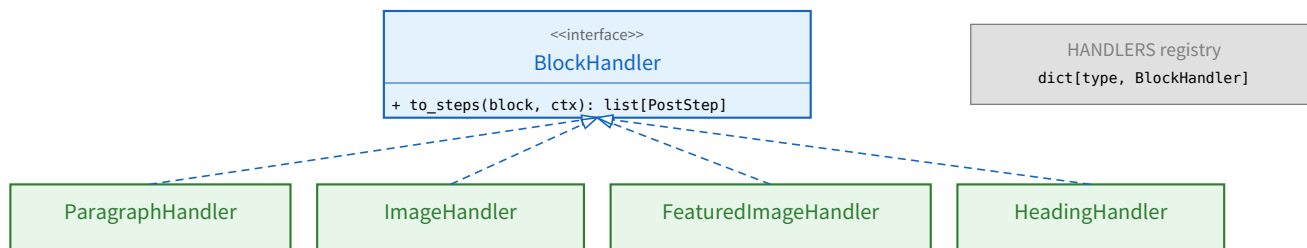


Fig 5-2. Strategy + Registry — PHP의 \$handlers[Block::class] = new Handler() 와 동일

PHP에서는 associative array \$handlers = [ParagraphBlock::class => new ParagraphHandler()]로 구현합니다. Python에서는 dict[type, BlockHandler]로, 키가 문자열이 아닌 타입 자체입니다. type(block)으로 조회하므로 isinstance 체인이 불필요합니다.

```
# automator/block_handlers.py
HANDLERS: dict[type[Block], BlockHandler] = {
    ParagraphBlock: ParagraphHandler(),
    ImageBlock: ImageHandler(),
    FeaturedImageBlock: FeaturedImageHandler(),
    # ...
}
```

```
}  
  
def get_handler(block: Block) -> BlockHandler:  
    return HANDLERS[type(block)] # O(1) lookup
```

5.3 Factory Method: create_block()

```
// PHP
// PHP Factory
class BlockFactory {
private const FACTORIES = [
    "paragraph" => ParagraphBlock::class,
    "image" => ImageBlock::class,
    // ...
];

public static function create(
    string $type, array $config
): Block {
    $cls = self::FACTORIES[$type];
    return new $cls(...$config);
}
```

```
# Python
# automator/block_factory.py
FACTORIES: dict[str, type] = {
    "paragraph": ParagraphBlock,
    "image": ImageBlock,
    "featured": FeaturedImageBlock,
    "heading": HeadingBlock,
    # ...
}

def create_block(
    block_type: str,
    config: dict[str, Any],
    values: dict[str, str] | None = None,
    keyword: str = "",
    media_resolver: Callable[[int], str] | None = None,
) -> Block:
    cls = FACTORIES[block_type]
    return cls(**_filter_fields(cls, config))
```

PHP new \$cls(...\$config) → Python cls(**config): 둘 다 문자열 키로 생성자 호출

6. Python 고유 패턴

6.1 제너레이터와 yield

```
// PHP
// PHP Generator
function fibonacci(): Generator {
    [$a, $b] = [0, 1];
    while (true) {
        yield $a;
        [$a, $b] = [$b, $a + $b];
    }
}
foreach (fibonacci() as $n) {
    if ($n > 100) break;
    echo $n;
}
```

```
# Python
# Python Generator
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

for n in fibonacci():
    if n > 100: break
    print(n)

# Generator expression (list comprehension lazy)
even = (x for x in range(100) if x % 2 == 0)
```

PHP Generator와 거의 동일, + generator expression으로 메모리 효율적 처리

6.2 컨텍스트 매니저 (with)

```
// PHP
// PHP - try/finally
$file = fopen("data.txt", "r");
try {
    $content = fread($file, filesize("data.txt"));
} finally {
    fclose($file);
}
```

```
# Python
# Python - with
with open("data.txt", "r") as file:
    content = file.read()
# file.close()

# automater
with pdfplumber.open("doc.pdf") as pdf:
    for page in pdf.pages:
        text = page.extract_text()
```

try/finally → with: 리소스 해제를 자동 보장

6.3 데코레이터

```
// PHP
// PHP Attribute (8.0+)
#[Route("/api/users")]
class UserController {
    #[Get("/")]
    public function list(): Response { ... }
}
```

```
# Python
# Python decorator
@app.get("/api/users")
async def list_users():
    return {...}

# @dataclass(frozen=True)
@dataclass(frozen=True)
class AccountOption:
    username: str
```

PHP Attribute → Python decorator: @로 시작, 함수/클래스를 변환하는 래퍼

@dataclass(frozen=True)는 클래스에 __init__, __eq__, __hash__, __repr__을 자동 생성하고 __setattr__을 금지하는 데코레이터입니다. PHP 8.2의 readonly class가 하는 일을 데코레이터 하나로 처리합니다.

6.4 Pathlib

```
// PHP
// PHP path handling
$path = __DIR__ . "/uploads/" . $id;
if (file_exists($path)) {
    $content = file_get_contents($path);
}
$ext = pathinfo($path, PATHINFO_EXTENSION);
```

```
# Python
# Python pathlib
from pathlib import Path
path = Path(__file__).parent / "uploads" / str(id)
if path.exists():
    content = path.read_bytes()
    ext = path.suffix # ".jpg"

# automater (factory/storage.py)
full_path = self.base_dir / rel_path
full_path.parent.mkdir(parents=True, exist_ok=True)
full_path.write_bytes(data)
```

/ 연산자로 경로 결합, .exists()/read_bytes()/suffix 등 OOP 인터페이스

6.5 Walrus 연산자 (:=)

<pre>// PHP // PHP - 문자열 파일 읽기 \$line = fgets(\$file); while (\$line !== false) { process(\$line); \$line = fgets(\$file); }</pre>	<pre># Python # Python 3.8+ - 문자열 while (line := file.readline()): process(line) # 문자열: 문자열을 문자열로 if (m := re.match(r"\{(\w+)\}", text)): token = m.group(1)</pre>
--	---

:= (walrus): 표현식 안에서 변수에 할당. PHP에는 대응 없음

6.6 구조적 패턴 매칭 (Python 3.10+)

PHP 8.0의 match는 값 비교만 가능하지만, Python의 match/case는 구조적 패턴 매칭을 지원합니다. 객체의 속성, 시퀀스의 구조, 타입을 동시에 매칭할 수 있습니다.

```
# 문자열 블록 - PHP 문자열 블록
match block:
case ParagraphBlock(keyword=kw) if kw: # 키워드 + 블록 + 블록
    prompt = build_seo_prompt(kw)
case ParagraphBlock(prompt=p): # 블록 + 블록
    prompt = p
case HeadingBlock(level=lv, text=t): # 블록 블록
    print(f"H{lv}: {t}")
case _: # default
    raise ValueError("Unknown block")

# 문자열 블록
match command:
case ["quit"]: action = "exit"
case ["go", direction]: action = f"move {direction}"
case ["get", *items]: action = f"pick up {items}"
```

automater는 현재 match/case를 사용하지 않고 dict lookup(HANDLERS, FACTORIES)을 씁니다. 이것은 Martin의 OCP 원칙에 더 부합합니다: 새 타입 추가 시 레지스트리에 등록만 하면 되고, match/case 블록을 수정할 필요가 없습니다.

6.7 Enum vs Literal — 언제 무엇을 쓰는가

<pre>// PHP // PHP 8.1 Enum enum Status: string { case Active = "active"; case Cooling = "cooling"; case Banned = "banned"; public function label(): string { return match(\$this) { self::Active => "active", self::Cooling => "cooling", self::Banned => "banned", }; } }</pre>	<pre># Python # Python Enum from enum import Enum class Status(str, Enum): ACTIVE = "active" COOLING = "cooling" BANNED = "banned" @property def label(self) -> str: return {"active": "active", "cooling": "cooling", "banned": "banned"}[self.value] # automater 문자열: Literal (문자열 문자열) Visibility = Literal["public", "private", "draft"] # Enum 문자열 문자열: JSON config 문자열 # "public" 문자열 문자열 문자열 문자열</pre>
---	---

PHP enum → Python Enum 또는 Literal. JSON 직렬화가 중요하면 Literal이 단순

6.8 __slots__: 메모리 최적화

```
# 문자열 문자열: 문자열 문자열 __dict__ (dict) 문자열 → 문자열 문자열 문자열
class Point:
    def __init__(self, x, y): self.x = x; self.y = y

# __slots__ 문자열: __dict__ 문자열 문자열 문자열 → 문자열 30-50% 문자열
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x, y): self.x = x; self.y = y

# dataclass + slots (Python 3.10+)
@dataclass(frozen=True, slots=True) # combo_generator.py 문자열 문자열
```

```
class ComboSpec:
    keyword_ids: list[int]
    spacing_rule_id: int
```

PHP에는 대응 개념이 없습니다. Python에서 수만 개의 객체를 생성할 때 (예: ComboSpec) slots=True로 메모리를 절약합니다.

7. SQLAlchemy 2.0 → Eloquent 매핑

7.1 모델 정의

```
// PHP
// Laravel Eloquent
class Account extends Model {
    protected $fillable = [
        "user_id", "platform_id",
        "username", "password_enc",
    ];

    protected $casts = [
        "extra" => "array",
        "last_used_at" => "datetime",
    ];

    public function owner(): BelongsTo {
        return $this->belongsTo(User::class);
    }
}
```

```
# Python
# factory/models.py
class Account(Base):
    __tablename__ = "accounts"

    id = mapped_column(Integer, primary_key=True)
    user_id = mapped_column(Integer,
        ForeignKey("users.id"), nullable=False)
    username = mapped_column(String(128))
    extra = mapped_column(JSON, nullable=True)
    last_used_at = mapped_column(DateTime)

    owner = relationship("User",
        back_populates="accounts")
```

Eloquent \$fillable → SQLAlchemy mapped_column, \$casts → 타입 자동, belongsTo → relationship

7.2 쿼리 빌더

```
// PHP
// Eloquent Query Builder
$accounts = Account::where("status", "active")
->where("last_used_at", "<", $cutoff)
->orderBy("last_used_at", "asc")
->get();
```

```
# Python
# SQLAlchemy 2.0 select()
from sqlalchemy import select

accounts = session.scalars(
    select(Account)
    .where(Account.status == "active")
    .where(Account.last_used_at < cutoff)
    .order_by(Account.last_used_at.asc())
).all()
```

Eloquent ::where() → SQLAlchemy select().where(): 체이닝 문법 유사

7.3 관계 (relationship)

```
// PHP
// Eloquent Relationships
class User extends Model {
    public function accounts(): HasMany {
        return $this->hasMany(Account::class);
    }
}

//
$user->accounts; // lazy loading
$user->load("accounts"); // eager
```

```
# Python
# SQLAlchemy
class User(Base):
    accounts = relationship("Account",
        back_populates="owner")

#
user.accounts # lazy loading
# eager loading:
session.scalars(
    select(User).options(
        joinedload(User.accounts)))
```

hasMany → relationship + back_populates: 양방향 관계 명시

M:N 관계

Eloquent: belongsToMany() + pivot table. SQLAlchemy: relationship(secondary=association_table). automater
실제: Combination.keywords = relationship('Keyword', secondary=combination_keywords). association table은
별도 Table() 객체로 정의합니다.

7.4 세션 생명주기: Laravel DB::transaction → Session

```
// PHP
// Laravel 테스트
DB::transaction(function () {
    $user = User::create([...]);
    Account::create([
        "user_id" => $user->id, ...
    ]);
}); // 테스트 commit or rollback
```

```
# Python
# SQLAlchemy 2.0 테스트
with Session(engine) as session:
    user = User(email="...", ...)
    session.add(user)
    session.flush() # SQL 실행, id 할당
    acc = Account(user_id=user.id, ...)
    session.add(acc)
    session.commit() # 테스트
# with 블록이 끝나면 session이 자동으로 close
```

DB::transaction → Session + commit/rollback, flush()는 SQL 실행하되 commit은 안 함

flush() vs commit(): flush()는 SQL을 실행하여 DB에 반영하지만 트랜잭션을 닫지 않습니다. commit()은 트랜잭션을 확정합니다. 테스트에서는 flush()만 쓰고 rollback()으로 격리합니다. PHP Doctrine의 \$em->flush()와 정확히 동일합니다.

7.5 테스트에서 DB 격리: RefreshDatabase → rollback

```
// PHP
// Laravel Test
class AccountTest extends TestCase {
    use RefreshDatabase; // 테스트마다 DB 초기화

    public function testCreate(): void {
        $user = User::factory()->create();
        $acc = Account::factory()->create([
            "user_id" => $user->id,
        ]);
        $this->assertDatabaseHas("accounts", [
            "username" => $acc->username,
        ]);
    }
}
```

```
# Python
# pytest + conftest.py (automater 테스트)
@pytest.fixture(scope="function")
def session(engine):
    with Session(engine) as sess:
        yield sess
        sess.rollback() # RefreshDatabase 테스트

def test_create(session):
    user = User(email="test@test.com", ...)
    session.add(user)
    session.flush()
    acc = Account(user_id=user.id, ...)
    session.add(acc)
    session.flush()
    assert acc.username == "test_user"
```

RefreshDatabase → session.rollback(): 마이그레이션 재실행보다 훨씬 빠름

automater의 테스트 격리 전략은 매우 효율적입니다: DB 스키마는 세션 당 한 번만 생성(scope="session")하고, 매 테스트 후 rollback으로 데이터를 원복합니다. Laravel의 RefreshDatabase가 매번 마이그레이션을 재실행하는 것보다 10-100배 빠릅니다.

7.6 Cascade와 생명주기 관리

```
// PHP
// Laravel - onDelete cascade
Schema::create("layout_slots", function ($t) {
    $t->foreignId("layout_id")
        ->constrained("post_layouts")
        ->onDelete("cascade");
});

// Eloquent relationship
public function slots(): HasMany {
    return $this->hasMany(LayoutSlot::class);
}
```

```
# Python
# automater 테스트 (factory/models.py)
class LayoutSlot(Base):
    layout_id = mapped_column(
        Integer,
        ForeignKey("post_layouts.id",
            ondelete="CASCADE"),
    )

class PostLayout(Base):
    slots = relationship(
        "LayoutSlot",
        back_populates="layout",
        order_by="LayoutSlot.sort_order",
        cascade="all, delete-orphan",
    )
```

onDelete('cascade') → ForeignKey(ondelete='CASCADE') + cascade='all, delete-orphan'

cascade="all, delete-orphan"은 ORM 레벨 cascade입니다. PostLayout을 삭제하면 연결된 LayoutSlot도 자동 삭제됩니다. ondelete="CASCADE"는 DB 레벨 cascade로, ORM을 우회한 직접 DELETE에도 동작합니다. automater는 양쪽 모두 설정하여 안전망을 이중화합니다.

7.7 async/await: PHP Fiber → Python asyncio

automater의 API 레이어는 FastAPI를 사용하며, FastAPI는 async/await를 지원합니다. PHP 8.1의 Fiber와 유사하지만 언어 수준에서 더 깊이 통합되어 있습니다.

<pre>// PHP // PHP 8.1 Fiber (Fiber) \$fiber = new Fiber(function (): void { \$value = Fiber::suspend("hello"); echo \$value; // "world" }); \$result = \$fiber->start(); // "hello" \$fiber->resume("world"); // Laravel HTTP (Fiber) Route::get("/users", function () { \$users = User::all(); return response()->json(\$users); });</pre>	<pre># Python # Python asyncio (Fiber) import asyncio async def fetch_data(): await asyncio.sleep(1) # non-blocking Fiber return "data" # FastAPI (automater API Fiber) from fastapi import FastAPI app = FastAPI() @app.get("/users") async def list_users(): users = await get_users() # async DB Fiber return users</pre>
--	---

PHP Fiber (저수준 코루틴) → Python async/await (언어 내장 코루틴)

automater에서 async를 쓰는 곳은 API 레이어뿐입니다. factory와 automator 레이어는 동기 코드입니다. BatchWorker가 Playwright(동기)를 사용하기 때문입니다. PHP에서 Laravel Queue가 동기 Job을 실행하는 것과 같은 구조입니다.

7.8 프로젝트 구조: Laravel → Python 패키지

Laravel	automater (Python)	역할
app/Models/	factory/models.py	ORM 모델 정의
app/Http/Controllers/	api/routes/*.py	HTTP 엔드포인트
app/Http/Requests/	api/schemas.py	요청 검증 (Pydantic)
app/Services/	factory/ + automator/	비즈니스 로직
app/Contracts/	automator/ports.py + contracts.py	인터페이스/계약
tests/Unit/	tests/unit/	단위 테스트
tests/Feature/	tests/integration/	통합 테스트
database/migrations/	(SQLAlchemy create_schema)	스키마 관리
config/	.env + automator/config.py	설정
routes/api.php	api/app.py	라우트 등록
composer.json	requirements.txt	의존성 관리
phpunit.xml	pytest.ini	테스트 설정
.env	.env + python-dotenv	환경 변수

Table 7-1. Laravel 프로젝트 구조 → automater Python 프로젝트 매핑

핵심 차이: Laravel은 app/ 아래 Models, Http, Services가 모두 모인 모놀리스 구조입니다. automater는 factory/, automator/, api/ 세 패키지가 물리적으로 분리되어 있으며, contracts.py가 유일한 접점입니다. 이 구조는 Laravel의 모놀리스보다 PHP Symfony의 Bundle 개념에 가깝습니다.

__init__.py의 역할

```
# 이 디렉토리 __init__.py는 Python 패키지 디렉토리
# PHP는 composer.json autoload PSR-4로 패키지 디렉토리
#
# automater-dev/
# automator/
# __init__.py ← 이 디렉토리 import automator.runner로
# runner.py
# ports.py
# factory/
# __init__.py
# models.py
# api/
# __init__.py
# app.py
# tests/
# __init__.py
# conftest.py ← fixture (PHP는 TestCase로)
```

__init__.py는 보통 빈 파일입니다. 존재 자체가 "이 디렉토리는 Python 패키지"라는 선언입니다. PHP에서 composer.json에 "autoload": {"psr-4": {"App\\": "app/"}}를 설정하는 것과 같습니다.

7.9 Python 3.12-3.14 최신 문법 하이라이트

automater는 Python 3.12+를 사용합니다. PHP 개발자가 알아야 할 최신 문법을 정리합니다.

버전	기능	코드 예시	PHP 대응
3.10	match/case (<code>match</code> 문법)	<pre>match block: case ParagraphBlock(kw=k): ...</pre>	match (단순 값 비교만)
3.10	X Y union 문법	<pre>def f(x: int str) -> None:</pre>	int string (동일)
3.10	ParamSpec / TypeVarTuple	<pre>def func(*args: ParamSpec, **kwargs: TypeVarTuple):</pre>	(없음)
3.10	dataclass(slots=True)	<pre>@dataclass(slots=True) class ComboSpec:</pre>	(없음) 메모리 최적화
3.12	f-string <code>{}</code> 문법	<pre>f"key='{name}'"</pre>	(PHP는 원래 가능)
3.12	type <code>[]</code> (PEP 695)	<pre>type Point = tuple[int, int]</pre>	PHPStan @psalm-type
3.12	<code>list[T]</code> 문법	<pre>def first[T](items: list[T]) -> T:</pre>	PHPStan @template
3.13	<code>no-GIL</code> 문법	<pre>python --free-threading</pre>	(해당 없음)
3.13	<code>JIT</code> 문법	<pre>python --jit (5-15% 향상)</pre>	PHP 8.0 JIT과 유사
3.14	Template Strings (PEP 750)	<pre>t"Hello {name}" (문법)</pre>	(없음)
3.14	annotations <code>from __future__ import annotations</code>	<pre>from __future__ import annotations</pre>	(해당 없음)

Table 7-2. Python 3.10-3.14 주요 신기능 — PHP 대응과 함께

automater에서 주의 automater는 Python 3.12+를 기준으로 작성되었습니다. X | Y 문법(3.10), dataclass(slots=True)(3.10), f-string 중첩 따옴표(3.12)는 자유롭게 사용됩니다. type 문(3.12)은 아직 사용하지 않고 Union[...]을 씁니다 — 하위 호환성 때문입니다. no-GIL과 JIT는 실험적이므로 프로덕션에서는 사용하지 않습니다.

8. 실전: automater 코드 읽기 가이드

이 장은 PHP 개발자가 automater 코드베이스를 처음 읽을 때 혼란스러운 패턴 10가지를 정리합니다.

코드 패턴	PHP 대응	의미
<code>from __future__ import annotations</code>	(<code>[]</code>)	타입 힌트를 문자열로 평가 (순환 참조 방지)
<code>@dataclass(frozen=True)</code>	<code>readonly class</code>	불변 데이터 객체, <code>__init__</code> 자동 생성
<code>field(default_factory=list)</code>	<code>= []</code>	가변 기본값을 매번 새로 생성 (Python 특유)
<code>ABC + @abstractmethod</code>	<code>interface</code>	구현을 강제하는 추상 타입
<code>Union[A, B] [] A B</code>	<code>A B (PHP 8.0)</code>	둘 중 하나의 타입
<code>Literal['a', 'b', 'c']</code>	<code>enum (enum [])</code>	허용 값 제한, JSON 직렬화 간편
<code>tuple[str, ...]</code>	<code>array (readonly)</code>	불변 시퀀스, 한번 생성 후 변경 불가
<code>dict[str, Any]</code>	<code>array<string, mixed></code>	키-값 매핑
<code>Callable[[int], str]</code>	<code>callable(int): string</code>	함수 타입 힌트
<code>if TYPE_CHECKING:</code>	(<code>[]</code>)	런타임에 import 안 함, 타입 체커만 참조
<code>session.scalars(select(X))</code>	<code>X::query()->...</code>	SQLAlchemy 2.0 쿼리 실행
<code>session.flush()</code>	<code>\$em->flush()</code>	SQL 실행 (commit 전), 테스트에서 rollback으로 격리
<code>SimpleNamespace(a=1, b=2)</code>	<code>(object)['a'=>1, 'b'=>2]</code>	동적 객체, 테스트 Fake에 사용
<code>Path(__file__).parent</code>	<code>__DIR__</code>	현재 파일의 디렉토리 경로

Table 8-1. automater 코드 읽기 치트시트 — PHP 대응과 함께

전체 흐름 시퀀스: PHP 개발자의 시선으로

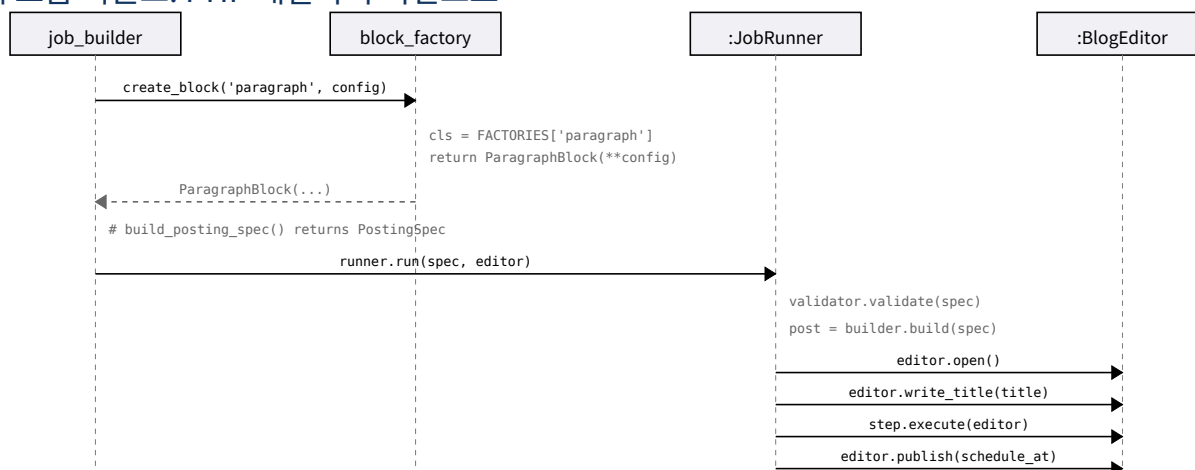


Fig 8-1. 전체 실행 흐름 — PHP 개발자를 위한 주석 포함

권장 읽기 순서

1. automator/options.py — 모든 frozen dataclass 정의. PHP의 readonly class/DTO에 해당. 여기서 전체 도메인 어휘를 파악.
2. automator/ports.py — ABC 3개. PHP interface와 1:1. 이것이 DIP의 경계.
3. automator/editor.py — BlogEditor ABC + PostStep 계층. PHP의 interface + Command Pattern.
4. automator/contracts.py — PostingSpec. 딱 5줄. 두 레이어의 유일한 접점.
5. automator/stubs.py — test double 3종. 테스트가 어떻게 동작하는지 이해.
6. tests/unit/automator/test_block_factory.py — 가장 단순한 단위 테스트. pytest 문법 학습.
7. tests/integration/automator/test_runner.py — RecordingEditor 패턴 학습.
8. factory/job_builder.py — DB→PostingSpec 변환. SQLAlchemy 패턴 학습.

자주 혼란스러운 코드 패턴 해설

automater 코드를 읽다가 멈춰하는 패턴들을 해설합니다.

패턴 1: from __future__ import annotations

```
# 00 00 0 00 00
from __future__ import annotations
# 000 000:
class A:
    def method(self) -> B: ... # NameError: B가 00 00 0 0
class B: ...
# 000 000:
# 00 000 0000 0000 00/00 00 00
```

패턴 2: if TYPE_CHECKING

```
# automator/block_handlers.py 00 00
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from automator.ports import TextGenerator, ImageProcessor
# -> 00000 import 0 0 (00 import 00)
# -> mypy/pyright가 0 import 0
# PHP 00: 00 (PHP가 autoload가 00 import 000 00)
```

패턴 3: field(default_factory=...)

```
# automator/options.py
@dataclass(frozen=True)
class PostingSpec:
    body: tuple[Section, ...] = () # tuple가 00000 00
    # 000 00 - list가 00000 default_factory 00
    # tags: list[str] = [] # 00! 00 00000 00 list 00
    tags: list[str] = field(default_factory=list) # 000
```

패턴 4: tuple[Section, ...]의 의미

```
# tuple[Section, ...] "Section가 00 00 00 000"
# ... (Ellipsis) "00 00" 00
# 00 00 00:
tuple[int, str] # 000 2: (1, "hello")
tuple[int, ...] # 00 00: (), (1,), (1,2,3)
tuple[()] # 0 000
# PHP 00: array<int, Section> (00 00 00 00)
```

패턴 5: | None vs Optional

```
# 0 00 000 00 00:
from typing import Optional
x: Optional[str] = None # 000
x: str | None = None # Python 3.10+ (automater 000)
x: Union[str, None] = None # 000
# PHP 00: ?string $x = null
```

종합 빠른 참조표

PHP와 Python의 핵심 대응을 한 눈에 정리합니다.

분류	PHP	Python
변수 선언	<code>\$name = 'hello';</code>	<code>name = 'hello'</code>
상수	<code>const F00 = 1; / define()</code>	<code>F00 = 1 ((): ())</code>
문자열 보간	<code>"Hello {\$name}"</code>	<code>f'Hello {name}'</code>
배열 → 리스트	<code>\$a = [1, 2, 3];</code>	<code>a = [1, 2, 3]</code>
연관 배열 → dict	<code>\$m = ['k' => 'v'];</code>	<code>m = {'k': 'v'}</code>
함수 정의	<code>function f(int \$x): int</code>	<code>def f(x: int) -> int:</code>
화살표 함수	<code>fn(\$x) => \$x * 2</code>	<code>lambda x: x * 2</code>
클래스 정의	<code>class Foo { ... }</code>	<code>class Foo: ...</code>
생성자	<code>__construct()</code>	<code>__init__(self, ...)</code>
\$this 참조	<code>\$this->name</code>	<code>self.name</code>
인터페이스	<code>interface I { ... }</code>	<code>class I(ABC): ...</code>
implements	<code>class A implements I</code>	<code>class A(I):</code>
extends	<code>class B extends A</code>	<code>class B(A):</code>
readonly class	<code>readonly class X { ... }</code>	<code>@dataclass(frozen=True)</code>
Nullable	<code>?string \$x</code>	<code>x: str None</code>
Union type	<code>int string</code>	<code>int str</code>
Enum	<code>enum E: string { ... }</code>	<code>class E(str, Enum): ...</code>
Exception	<code>throw new E()</code>	<code>raise E()</code>
Try/catch	<code>try { } catch (E \$e)</code>	<code>try: ... except E as e:</code>
Namespace	<code>namespace App\X;</code>	<code># ==, =====</code>
Import	<code>use App\X\Foo;</code>	<code>from app.x import Foo</code>
Array map	<code>array_map(fn, \$arr)</code>	<code>[fn(x) for x in arr]</code>
Array filter	<code>array_filter(\$arr, fn)</code>	<code>[x for x in arr if fn(x)]</code>
Null coalesce	<code>\$x ?? 'default'</code>	<code>x if x is not None else 'default'</code>
Spread	<code>...\$args</code>	<code>*args, **kwargs</code>
Type check	<code>instanceof</code>	<code>isinstance(obj, cls)</code>
Ternary	<code>\$a ? \$b : \$c</code>	<code>b if a else c</code>
코멘트	<code>// /* */</code>	<code># """ docstring """</code>
패키지 관리	<code>composer</code>	<code>pip (requirements.txt)</code>
테스트	<code>PHPUnit</code>	<code>pytest</code>
ORM	<code>Eloquent / Doctrine</code>	<code>SQLAlchemy 2.0</code>
Web 프레임워크	<code>Laravel / Symfony</code>	<code>FastAPI / Django</code>

Table 8-2. PHP → Python 종합 빠른 참조표

9. 연습 문제

각 문제는 PHP 코드를 Python으로 변환하는 형식입니다. automater 코드베이스의 패턴을 사용합니다.

연습 1: PHP interface → Python ABC

다음 PHP 인터페이스를 Python ABC로 변환하시오:

```
// PHP
interface MediaStorage {
    public function save(int $userId, string $filename, string $data): string;
    public function read(string $relPath): string;
    public function delete(string $relPath): void;
}
```

정답:

```
# Python
from abc import ABC, abstractmethod

class MediaStorage(ABC):
    @abstractmethod
    def save(self, user_id: int, filename: str, data: bytes) -> str: ...

    @abstractmethod
    def read(self, rel_path: str) -> bytes: ...

    @abstractmethod
    def delete(self, rel_path: str) -> None: ...
```

참고: PHP의 string \$data는 Python에서 bytes입니다 (바이너리 파일). automater의 LocalStorage.save()가 이 패턴을 사용합니다.

연습 2: PHP readonly class → frozen dataclass

다음 PHP readonly class를 Python frozen dataclass로 변환하시오:

```
// PHP
readonly class ScheduleConfig {
    public function __construct(
        public string $mode = "immediate",
        public ?DateTimeImmutable $at = null,
        public int $jitterMinutes = 30,
        public array $tags = [],
    ) {}
}
```

정답:

```
# Python
from dataclasses import dataclass, field
from datetime import datetime
from typing import Literal

ScheduleMode = Literal["immediate", "fixed", "random_window"]

@dataclass(frozen=True)
class ScheduleConfig:
    mode: ScheduleMode = "immediate"
    at: datetime | None = None
    jitter_minutes: int = 30
    tags: list[str] = field(default_factory=list)
```

주의: tags의 기본값은 field(default_factory=list)여야 합니다. = []로 쓰면 모든 인스턴스가 같은 리스트를 공유하는 버그가 발생합니다.

연습 3: PHPUnit test → pytest

다음 PHPUnit 테스트를 pytest로 변환하시오:

```
// PHP
class TitleGeneratorTest extends TestCase {
    public function testFixedTitleReturnsAsIs(): void {
        $option = new TitleOption(fixedTitle: "Hello World");
        $result = generateTitle($option);
        $this->assertEquals("Hello World", $result);
    }
}
```

```
public function testEmptyTemplateRaisesIfSalts(): void {
    $option = new TitleOption(
        template: "",
        prefixSalts: ["a"],
    );
    $this->expectException(ValueError::class);
    generateTitle($option);
}
}
```

정답:

```
# Python
import pytest
from automator.options import TitleOption
from automator.title_generator import generate_title

def test_fixed_title_returns_as_is():
    option = TitleOption(fixed_title="Hello World")
    assert generate_title(option) == "Hello World"

def test_empty_template_raises_if_salts():
    option = TitleOption(
        template="",
        prefix_salts=("a",), # tuple!
    )
    with pytest.raises(ValueError):
        generate_title(option)
```

주의: PHP의 camelCase(fixedTitle) → Python의 snake_case(fixed_title). PHP의 배열 ['a'] → Python의 tuple ('a',) — 심포 필수, 없으면 문자열 'a'가 됩니다.

연습 4: Eloquent → SQLAlchemy 쿼리

다음 Eloquent 쿼리를 SQLAlchemy 2.0으로 변환하시오:

```
// PHP
$active_accounts = Account::where("status", "active")
->where("platform_id", $platformId)
->whereNull("last_used_at")
->orWhere("last_used_at", "<", $cutoff)
->orderBy("last_used_at", "asc")
->get();
```

정답:

```
# Python
from sqlalchemy import select, or_
from factory.models import Account

active_accounts = session.scalars(
    select(Account)
    .where(
        Account.status == "active",
        Account.platform_id == platform_id,
        or_(
            Account.last_used_at.is_(None),
            Account.last_used_at < cutoff,
        ),
    )
    .order_by(Account.last_used_at.asc())
).all()
```

주의: whereNull → .is_(None) (is_에 언더스코어!), orWhere → or_() 함수로 감싸기, 여러 where 조건은 심포로 연결 (암묵적 AND).

연습 5: Strategy 패턴 — BlockHandler 추가

새로운 EmbedBlock(url: str, provider: str) 블록을 추가하시오. 이 블록은 YouTube/Vimeo 임베드를 나타내며, BlogEditor.insert_text()로 iframe HTML을 삽입합니다. 필요한 모든 파일의 변경사항을 나열하시오.

정답 요약:

파일

변경

automator/options.py	@dataclass(frozen=True) class EmbedBlock: url, provider Block Union[] []
automator/editor.py	@dataclass(frozen=True) class EmbedStep(PostStep): execute → editor.insert_text(iframe_html)
automator/block_handlers.py	class EmbedHandler(BlockHandler): to_steps() HANDLERS[EmbedBlock] = EmbedHandler()
automator/block_factory.py	FACTORIES["embed"] = EmbedBlock
tests/unit/...	test_block_factory: test_embed_from_config() test_all_block_types_registered: 'embed' []

이 패턴은 UML 가이드의 7.1절 "새 Block 타입 추가 체크리스트"와 동일합니다. OCP 준수: ContentBuilder와 JobRunner는 수정하지 않습니다.

이 교재는 Martin의 조언을 따라 "다이어그램에서 코드를 떠올릴 수 있어야 한다"는 원칙으로 작성되었습니다. 모든 UML 다이어그램은 automater 코드베이스의 실제 클래스에서 가져왔으며, 모든 PHP↔Python 비교는 동일한 의미를 가지는 실제 코드입니다.